A Typed Agent Action Framework with Prefix-Verdict Semantics

Technical Report — Version 1

Davide Bragetti*
Independent Researcher, Rome, Italy

Giuseppe Francesco Italiano
LUISS Guido Carli, Rome, Italy

2026-08-21

Abstract

We present a compact typed framework for reasoning about agent actions against temporal properties evaluated on finite observation prefixes. Five footprint action classes at layer L_1 — READ, THINK, SPEAK, WRITE, ACT — are typed by their read/write footprints on a partitioned global state, and their epistemic consequences are evaluated via the standard three-valued prefix-verdict semantics of runtime verification. From this combination we derive a small but honest structural signature: READ is the unique L_1 footprint class that can directly change an agent's prefix verdict; cross-agent verdict effects must factor through a recipient-side READ; the space of effect states collapses to three reachable pairs $(0, \mathrm{N/A}), (+,+), (+,-)$, showing that effect notation is derivable from verdict transitions rather than an independent primitive. A framework boundary is stated explicitly: the existence of a finite certificate for a property does not imply that an agent will acquire it.

This is a typed integration interface over standard runtime- verification and process-calculus machinery, not a new formal theory. Every structural corollary is proved in a few lines from standard non-interference and message-passing facts together with the framework's explicit dependency of verdicts on observed prefixes alone.

1 Introduction

Agent frameworks routinely distinguish sensing from thinking from communication from action, but the distinctions are often carried by prose rather than by types. This paper offers a compact interface that types those distinctions and connects them to a standard temporal-property verdict semantics.

Two separations we make explicit. The framework rests on two distinctions that agent-framework prose sometimes elides.

World truth is not agent-accessible verdict. An ω-behavior w either satisfies a temporal property φ or does not: $w \models \varphi$ or $w \not\models \varphi$. An agent observing a finite prefix u of w has access to a three-valued verdict $V_\varphi(u) \in \{T, F, U\}$ (true, false, unknown). The verdict is a prefix classification, not an ontological truth value; UNKNOWN means the finite evidence does not yet force satisfaction or violation, and no world-side "third truth" is invoked. This is the standard Bauer–Leucker–Schallhart three-valued finite-prefix reading [4][1].

Certificate existence is not certificate acquisition. A property may possess a finite certificate — a good prefix, a bad prefix — in every satisfying (resp. violating) trace, and yet the agent may fail to observe such a prefix if its READ schedule does not cover the certificate-bearing events.

*Corresponding author: bragettidavide@gmail.com

[1] We use the three-valued semantics of Bauer, Leucker and Schallhart [4] throughout, not the four-valued RV-LTL refinement compared in their earlier work [3]; "RV-LTL" below refers to the three-valued version.

This gap is a scope-level boundary of the framework, not a defect, and is stated explicitly (Section 10).

What the framework is not. This paper does not introduce a new temporal logic. It does not claim that five action classes are primitive, minimal, or complete in a strong formal sense. It does not commit to a causal (interventional) sign for actions; several candidate constructions were considered and none was clean enough at the intended foundational scope. It is a compact interface, and its main results are structural: they follow in one or two lines from footprint semantics plus a single design choice about how verdicts depend on agent state.

Structure. Section 2 fixes notation. Section 3 defines the global-state model and the minimal agent state. Section 4 introduces the five footprint action classes at layer L_1 as footprint equivalence classes. Section 5 adopts the RV-LTL prefix-verdict semantics. Section 6 proves verdict non-interference and the READ direct verdict gate. Section 7 factorises cross-agent verdict effects. Section 8 identifies the three reachable effect states and demotes the (I, R) notation to derived. Section 9 tabulates property-class × verdict-mobility. Section 10 states Framework Boundary 1. Section 11 places the framework against related work; Section 14 lists limitations.

2 Preliminaries

Let Σ be a fixed finite alphabet of events. Σ^* denotes finite words, Σ^ω denotes infinite words, and $u \sqsubseteq w$ denotes prefix relation. A *temporal property* is a set $\varphi \subseteq \Sigma^\omega$. We adopt the Alpern–Schneider safety–liveness dichotomy [1] and the runtime-verification four-class canon (Safety, Cosafety, Liveness, Coliveness) as formalised by Bauer–Leucker–Schallhart [4], based on the good-prefix / bad-prefix reading of Kupferman–Vardi [12]. The Manna–Pnueli hierarchy [13] is the ancestor of the general idea of a temporal-property hierarchy; the four-class canon we use is not identical to Manna–Pnueli's six-class hierarchy and we do not attribute it to them.

Definition 2.1 (Good/bad prefixes)**.** For $\varphi \subseteq \Sigma^\omega$,

$$\mathrm{Good}(\varphi) = \{u \in \Sigma^* \mid \forall \tau \sqsupseteq u.\ \tau \in \varphi\}, \quad \mathrm{Bad}(\varphi) = \{u \in \Sigma^* \mid \forall \tau \sqsupseteq u.\ \tau \notin \varphi\}.$$

Definition 2.2 (Property classes [1, 4, 12])**.** A property φ is

- *safety* (in S) iff every violation has a bad-prefix witness: $\forall \sigma \notin \varphi.\ \exists \pi \sqsubseteq \sigma.\ \pi \in \mathrm{Bad}(\varphi)$;
- *cosafety* (in C) iff every satisfaction has a good-prefix witness: $\forall \sigma \in \varphi.\ \exists \pi \sqsubseteq \sigma.\ \pi \in \mathrm{Good}(\varphi)$;
- *liveness* (in L) iff $\mathrm{Bad}(\varphi) = \emptyset$;
- *coliveness* (in K) iff $\mathrm{Good}(\varphi) = \emptyset$.

3 Global state and agent model

Fix a finite set $\{1, \ldots, n\}$ of agents.

Definition 3.1 (Global state)**.** A *global state* is a tuple

$$z = (E,\ X_1, \ldots, X_n,\ M,\ P)$$

where E is object-level environmental state, X_i is the internal state of agent i, M is a *transient* communication state, and P is a *persistent* informational state. Under a declared protocol layer L_1 we require M, P, and E pairwise disjoint as state components.

Definition 3.2 (Agent state)**.** The internal state of agent i is $X_i = (u_i, \eta_i)$, where $u_i \in \Sigma_i^*$ is the observed prefix of the alphabet Σ_i available to agent i, and $\eta_i \in H_i$ is an opaque internal-state parameter. Knowledge, goals, capabilities and policies live in extension layers and are not part of the foundational core.

Definition 3.3 (Observation model)**.** An *observation model* $O = (\textit{fidelity}, \textit{invasiveness})$ declares the coupling between agent observations and world events. The foundational core is stated under $O_{\text{default}} = (\textit{perfect}, \textit{non-invasive})$: when a READ fires the resulting observation is faithful and does not perturb E. Extensions to imperfect or invasive regimes are deferred.

Definition 3.4 (Layer abstraction)**.** An abstraction layer L carries a partition of a state space Z_L into components, and a projection $\rho_L : Z \to Z_L$ from the global state onto the layer's components. Actions are typed relative to a declared layer.

4 Canonical action classes

Let T_L denote the set of typed transitions on Z_L. Each action occurrence induces read/write footprints on layer-L components.

Definition 4.1 (Footprints and equivalence)**.** For an action occurrence a inducing $z \to z'$, let $\mathrm{Rd}(a) \subseteq \text{Components}(Z_L)$ record the components a consults and $\mathrm{Wr}(a) \subseteq \text{Components}(Z_L)$ record the components a modifies. Define layer-L footprint equivalence $\sim_L$ on T_L by

$$a \sim_L b \iff (\mathrm{Rd}_L(a), \mathrm{Wr}_L(a)) = (\mathrm{Rd}_L(b), \mathrm{Wr}_L(b)).$$

Definition 4.2 (The five footprint action classes at layer L_1)**.** At layer L_1 under O_{default}, we distinguish the following L_1-footprint $\sim_{L_1}$-equivalence classes distinguished by this framework, referred to by the names R_i, T_i, $S_i(j)$, W_i, A_i for agent i (and receiver j for SPEAK):

Class	Rd	Wr
R_i	$\{E, M, P\}$	$\{X_i\}$
T_i	$\{X_i\}$	$\{X_i\}$
$S_i(j)$	$\{X_i\}$	$\{M\}$
W_i	$\{X_i\}$	$\{P\}$
A_i	$\{X_i, E\}$	$\{E\}$

An action occurrence that writes more than one distinct component of Z_{L_1} is *composite* and factors into a sequence of L_1-footprint single-write occurrences.

Remark 4.3 (Canonical, not primitive)**.** The five classes are *framework-local names* for five pairwise-distinct $\sim_{L_1}$-equivalence classes. We do not claim that they form a basis, that five is minimal, or that they cover every possible transition in every possible layer. Finer decompositions of READ by source (environment, message, persistent record) are natural at layers below L_1; coarser decompositions parameterising communication by persistence are natural at layers above.

Remark 4.4 (Footprint uniqueness at L_1, by inspection)**.** Under the L_1 pairwise-disjointness of $\{M, P, E\}$, the five footprint tuples of Definition 4.2 are pairwise distinct. Every pair is separated by either a difference in read set or a difference in write set. By inspection of the ten unordered pairs: only R_i and T_i share write set ($\{X_i\}$); their read sets differ (R_i admits E, M, P; T_i does not); all other pairs differ on write set.

Convention 4.5 (Protocol and policy)**.** A *protocol* $\Gamma \subseteq (\mathcal{A} \times \{1, \ldots, n\})^*$ is a language of legal finite executions; it is a specification device, not a transition system. A *policy* for agent i is a map $\pi_i : X_i \to \mathcal{P}(\mathcal{A})$ from the agent's internal state to the set of actions it enables; policies see only X_i, not the global state. Ontology, protocol, and policy remain distinct throughout.

5 Prefix-verdict semantics

We adopt the three-valued prefix-verdict function of RV-LTL verbatim [4]. For $\varphi \subseteq \Sigma^\omega$ and $u \in \Sigma^*$,

$$V_\varphi(u) = \begin{cases} T & \text{if } u \in \mathrm{Good}(\varphi), \\ F & \text{if } u \in \mathrm{Bad}(\varphi), \\ U & \text{otherwise.} \end{cases}$$

Agents evaluate this function on their observed prefixes: $V_\varphi(u_i)$ is agent i's verdict for φ at the current state. *The verdict function depends only on u_i*, which becomes the pivotal design choice for the results in Section 6.

Theorem 5.1 (Prefix verdict monotonicity, standard [4])**.** *For every $\varphi \subseteq \Sigma^\omega$ and every $u, u' \in \Sigma^*$ with $u \sqsubseteq u'$,*

$$V_\varphi(u) = T \implies V_\varphi(u') = T, \qquad V_\varphi(u) = F \implies V_\varphi(u') = F.$$

If $V_\varphi(u) = U$ then $V_\varphi(u') \in \{T, F, U\}$.

Proof. If $u \in \mathrm{Good}(\varphi)$, every $\tau \sqsupseteq u$ satisfies φ; in particular every $\tau \sqsupseteq u'$ does, so $u' \in \mathrm{Good}(\varphi)$ and $V_\varphi(u') = T$. Symmetrically for Bad. The U case follows by exhaustion. □

Remark 5.2 (UNKNOWN is not a truth value)**.** U is a *prefix classification*, not a third truth value about w. It records that neither Good nor Bad closure has been reached on the finite evidence so far. In particular, for any liveness $\varphi \in \mathsf{L}$ we have $\mathrm{Bad}(\varphi) = \emptyset$, so no finite u ever attains $V_\varphi(u) = F$; nevertheless φ may fail on some w, and system-side certificates (ranking functions, inductive invariants, Büchi acceptance witnesses, strategies) remain outside the prefix-classification taxonomy.

6 Verdict non-interference and the READ gate

The framework's structural spotlight begins with a standard observation from dependency analysis and non-interference: if an action's write footprint is disjoint from the components a derived observable depends on, the observable is preserved. We restate it in this setting.

Definition 6.1 (Dependency)**.** For a derived observable $f : Z_L \to Y$, let $\mathrm{Dep}(f) \subseteq \mathrm{Components}(Z_L)$ be a set of components such that $f(z)$ is a function of the restriction $z|_{\mathrm{Dep}(f)}$. We write "f depends only on $\mathrm{Dep}(f)$".

Theorem 6.2 (Verdict non-interference by footprint, standard)**.** *Let $f : Z_L \to Y$ be a derived observable depending only on $\mathrm{Dep}(f)$, and let a be an action inducing $z \to z'$. If $\mathrm{Wr}_L(a) \cap \mathrm{Dep}(f) = \emptyset$, then $f(z) = f(z')$.*

Proof. If a writes no component in $\mathrm{Dep}(f)$, then $z|_{\mathrm{Dep}(f)} = z'|_{\mathrm{Dep}(f)}$, so $f(z) = f(z')$. □

The verdict function satisfies the hypothesis: V_φ evaluated by agent i depends only on u_i, which is a component of X_i. The following is then an immediate corollary of Theorem 6.2 together with the framework's design choice Definition 3.2 to restrict V_φ's dependency to u_i. It is standard in the sense of the "knowledge is a function of local view" discipline of interpreted systems [10]; we state it here because it names a distinguished role for one of the five action classes.

Remark 6.3 (READ direct verdict gate, immediate corollary of def:xi + def:actions)**.** Under O_{default} and layer L_1, if $z \to z'$ under an action $a \in \{T_i, S_i(j), W_i, A_i\}$, then $V_\varphi(u_i(z)) = V_\varphi(u_i(z'))$ for every temporal property φ. Consequently, at layer L_1 the only L_1-footprint action class that can *directly* change agent i's verdict on φ is R_i. Take $f(z) := V_\varphi(u_i(z))$. By Definition 3.2, u_i is a component of X_i; $\mathrm{Dep}(f) = \{X_i^{[u_i]}\}$, the u_i-slot of X_i. The write sets of T_i, $S_i(j)$, W_i, A_i per Definition 4.2 are $\{X_i\}, \{M\}, \{P\}, \{E\}$; only T_i's write set shares a top-level component with $\mathrm{Dep}(f)$. But T_i writes $X_i^{[\eta_i]}$ and does not touch u_i: T_i takes as input X_i and produces a new η_i, leaving u_i unchanged (Definition 3.2 plus the type discipline that only READ extends the observation record). Hence $\mathrm{Wr}_L(a) \cap \mathrm{Dep}(f) = \emptyset$ for all four cases, and Theorem 6.2 yields $V_\varphi(u_i(z)) = V_\varphi(u_i(z'))$.

Remark 6.4 (Direct vs mediated)**.** Remark 6.3 concerns a *single-step, single-agent* verdict transition. Actions in $\{T_i, S_i(j), W_i, A_i\}$ can still change future verdicts through their *mediated* effect on subsequent observations: a W_i writes to P, which a later R_k (any k) may read, potentially updating u_k and hence $V_\varphi(u_k)$. The distinction between direct and mediated verdict effect is a consequence of the footprint semantics, not an additional axiom.

7 Cross-agent verdict effects

The single-agent statement generalises straightforwardly to inter-agent verdict effects. Since agent j's verdict depends only on u_j, and only R_j writes to u_j, any change in $V_\varphi(u_j)$ resulting from an action by agent $i \neq j$ must factor through a subsequent R_j. This is the standard send / receive factorisation of process calculi [14, 11], specialised at layer L_1 by the (transient, persistent) carrier split (M, P); we record the statement as a corollary rather than as a novel theorem.

Remark 7.1 (Cross-agent composite requirement, immediate corollary)**.** Under O_{default} and layer L_1, let a be a single action by agent i inducing $z \to z'$, and let $j \neq i$. Then $V_\varphi(u_j(z)) = V_\varphi(u_j(z'))$ for every φ. In particular any cross-agent verdict effect from i to j must occur through a composite $a; R_j$ (possibly with intervening steps). By Definition 4.2 no L_1-footprint action by agent i has u_j in its write set. Apply Theorem 6.2 with $f = V_\varphi \circ u_j$ and $\mathrm{Dep}(f) = \{X_j^{[u_j]}\}$.

Remark 7.2 (Two canonical communication composites)**.** Inter-agent verdict transmission at layer L_1 factors as either $S_i(j); R_j$ (transient) or $W_i; R_k$ (persistent), depending on whether the intermediate content lives in M or P. Under protocol declarations that bound M's lifetime, $S_i(j); R_j$ is feasible only within the declared window; $W_i; R_k$ is feasible at any subsequent step. Direct from Definition 4.2 write sets and Remark 7.1.

7.1 Reflexive reentry, and where the two carriers differ

Remark 7.1 is stated for $j \neq i$ and its proof uses that no L_1-footprint action by i writes u_j. That argument is unavailable when $j = i$, since R_i does write X_i. The reflexive case is therefore not a specialisation of Remark 7.1; it rests on Remark 6.3 instead, and it is worth recording because the two carriers behave differently there.

No new action class is needed. The receiver index of $S_i(j)$ does not occur in its footprint $(\{X_i\}, \{M\})$, so $S_i(i)$ is the same $\sim_{L_1}$-class as any other SPEAK; and W_i writes P, which R_i reads. Both $S_i(i); R_i$ and $W_i; R_i$ are well-formed composites of Definition 4.2 as it stands.

Remark 7.2 appeals to "protocol declarations that bound M's lifetime" without defining one. We make that proviso explicit, since the statement below is carried entirely by it.

Definition 7.3 (Carrier lifetime and availability)**.** A protocol declaration assigns each carrier $k \in \{E, M, P\}$ a *lifetime* $L_k \in \mathbb{N} \cup \{\infty\}$. Content deposited in k at step t is *available* to a READ

at step t' iff $t' - t \leq L_k$. A carrier is *transient* if $L_k < \infty$ and *persistent* if $L_k = \infty$. At layer L_1 the intended reading of the (M, P) split is $L_M < \infty$ and $L_P = \infty$, but nothing below assumes it: the lifetimes are parameters.

Call a payload c *dually representable* if a READ of its deposit in M and a READ of its deposit in P append the same symbol σ_c to u_i. We assume this only of the single payload used below; no general correspondence between spoken and written content is claimed.

Proposition 7.4 (Reflexive reentry is carrier-sensitive)**.** *Fix an agent i, a dually representable payload c, and a property φ with $V_\varphi(u) = U$ and $V_\varphi(u\sigma_c) = T$. Write $\rho_k(\Delta)$ for the run that deposits c in carrier k, idles for $\Delta - 1$ steps, and then performs R_i. Under Definition 7.3 with $L_M = L$ and $L_P = \infty$:*

(i) for $\Delta \leq L$, both $\rho_M(\Delta)$ and $\rho_P(\Delta)$ end with $V_\varphi(u_i) = T$;

(ii) for $\Delta > L$, $\rho_P(\Delta)$ ends with $V_\varphi(u_i) = T$ while $\rho_M(\Delta)$ ends with $V_\varphi(u_i) = U$.

The inclusion in (ii) is therefore strict. Both clauses fail if the lifetimes are exchanged: with $L_M = \infty$ the two runs agree at every Δ, and with $L_P < \Delta$ neither reaches T.

Proof. By Remark 6.3 neither the depositing action ($S_i(i)$ or W_i) nor any idling step changes $V_\varphi(u_i)$: of the five classes only R_i writes u_i. So the verdict at the end of each run is determined by what the final R_i appends. By Definition 7.3 the deposit is available to that READ iff $\Delta \leq L_k$. For $k = P$ this holds for every Δ; for $k = M$ it holds exactly when $\Delta \leq L$. When the deposit is available, R_i appends σ_c and the verdict is T by hypothesis; when it is not, u_i is unchanged and the verdict remains U. Strictness is $T \neq U$. The two failure clauses follow by substituting the stated lifetimes into the same availability condition. □

Remark 7.5 (What this does and does not say)**.** The proposition is a statement about carrier lifetimes, not about speaking and writing. Its content is that an agent can act on its own externalised output only while that output is still readable, and that the two carriers of Definition 3.1 bound that window differently. SPEAK may support immediate reflexive feedback; WRITE additionally supports delayed reflexive feedback, because the inscription outlives the utterance. Nothing here claims that speaking changes the speaker, that writing is cognitively stronger, or that inscription creates commitment. The two failure clauses are the guard: exchange the lifetimes and the asymmetry reverses, which it could not do if it were a property of the action names.

Remark 7.6 (The same schema, other carriers)**.** Both $S_i(i); R_i$ and $W_i; R_i$ instantiate one schema: deposit in a carrier k, then R_i. The two cases are $k = M$ and $k = P$. $A_i; R_i$ has the same shape with $k = E$, and Proposition 7.4 would apply to it verbatim under a declared L_E. We do not develop that case: unlike M and P, the environment is written by other agents and by the world, so availability there is not governed by a lifetime alone. We record the generalisation and stop.

8 Effect state space and derived notation

Under Theorem 5.1 the pre/post pair $(V_\varphi(u_i(z)), V_\varphi(u_i(z')))$ can take at most five values: $(T,T), (F,F), (U,U), (U,T), (U,F)$. The remaining four values — (T,U), (T,F), (F,U), (F,T) — are excluded by monotonicity. We record this as a small structural consequence and use it to demote a piece of notation from primitive to derived.

Some earlier drafts of this framework introduced a two-axis effect tuple

$$\mathrm{eff}_\varphi(a, z) = (I_\varphi, R_\varphi)$$

with $I_\varphi \in \{0, +\}$ recording whether a verdict resolution occurred and $R_\varphi \in \{+, -, \mathrm{N/A}\}$ recording the valence of the newly resolved verdict.

Remark 8.1 (Effect state space collapse, by enumeration)**.** Under Theorem 5.1, the reachable pairs (I_φ, R_φ) for any action a and property φ are exactly

$$\{(0, \mathrm{N/A}),\ (+,+),\ (+,-)\}.$$

The nominal $2 \times 3 = 6$-cell effect space collapses to three states. In particular the tuple (I_φ, R_φ) is a function of the verdict transition $(V_\varphi(u_i), V_\varphi(u_i'))$. By enumeration of the five monotone-legal verdict transitions of Theorem 5.1: for (T,T), (F,F), (U,U), $I_\varphi = 0$ (no resolution occurred) and $R_\varphi = \mathrm{N/A}$; for (U,T), $I_\varphi = +, R_\varphi = +$; for (U,F), $I_\varphi = +, R_\varphi = -$. No other transitions are reachable.

Remark 8.2 (Demotion)**.** Remark 8.1 implies that the effect tuple carries no information beyond the verdict transition itself. We accordingly *do not* treat (I_φ, R_φ) as a foundational primitive of the framework; we keep it available as expository notation for verdict transitions but drop it from the underlying scope contract. The framework's structural content lives entirely in Definitions 2.1, 2.2, 3.1 to 3.3 and 4.2.

9 Property-class asymmetries

Prefix-verdict mobility differs sharply across Alpern–Schneider–Manna–Pnueli property classes. The relevant facts are direct from Definition 2.2 and the certificate reading of the four classes established in the programme's programme certificate bridge.

Theorem 9.1 (Prefix-verdict class asymmetry, standard-derived)**.** *For every $\varphi \subseteq \Sigma^\omega$ and every $u \in \Sigma^*$,*

(i) if $\varphi \in \mathsf{L}$ (so $\mathrm{Bad}(\varphi) = \emptyset$), then $V_\varphi(u) \neq F$;

(ii) if $\varphi \in \mathsf{K}$ (so $\mathrm{Good}(\varphi) = \emptyset$), then $V_\varphi(u) \neq T$;

(iii) if $\varphi \in \mathsf{L} \cap \mathsf{K}$, then $V_\varphi(u) = U$ for every u.

Proof. $V_\varphi(u) = F$ requires $u \in \mathrm{Bad}(\varphi)$; for $\varphi \in \mathsf{L}$ this set is empty, giving (i). Symmetrically (ii). Case (iii) is the conjunction. □

Combined with Remark 6.3, Theorem 9.1 yields the following sharp form of "information capability" per class.

Corollary 9.2 (Read is powerless on $\mathsf{L} \cap \mathsf{K}$)**.** *If $\varphi \in \mathsf{L} \cap \mathsf{K}$, then for every L_1-footprint action a (including R_i) applied at any global state, $V_\varphi(u_i)$ remains U. No finite prefix can resolve φ.*

Proof. By Theorem 9.1(iii) the verdict is U regardless of how the prefix is extended. □

Corollary 9.2 is the sharpest form in which the framework's action-typing layer meets the temporal-class layer. It should be read alongside the discipline that *prefix certificates are not the only certificates*: for a property in L, absence of a finite refutation certificate does not preclude a system-side proof of the property from a ranking function or from a Büchi acceptance witness.

Class	$U \to T$	$U \to F$	T guaranteed on $w \models \varphi$?	F guaranteed on $w \not\models \varphi$?
S	iff Good $\neq \emptyset$	possible	no (in general)	yes
C	possible	iff Bad $\neq \emptyset$	yes	no (in general)
L	iff Good $\neq \emptyset$	impossible	no	no
K	impossible	iff Bad $\neq \emptyset$	no	no
$\mathsf{S} \cap \mathsf{C}$	possible	possible	yes	yes
$\mathsf{L} \cap \mathsf{K}$	impossible	impossible	no	no

10 Framework boundary: existence versus acquisition

The observation model Definition 3.3 guarantees only that observations are *faithful when* READ *fires*; it does not guarantee that READ fires often enough to observe each certificate- bearing event. The underlying mathematical content is a *diagnosability* failure in the sense of Sampath, Sengupta, Lafortune, Sinnamohideen, and Teneketzis [19]: a projection observer may fail to detect a global-trace event within any finite delay. Decentralised runtime verification (Bauer and Falcone [2]) addresses the same question for LTL monitors under partial observation. The contribution here is to state this gap as a *first-class scope boundary* of the framework, so downstream extensions can name the observation- coverage assumption they require.

Proposition 10.1 (Certificate existence is not certificate acquisition)**.** *Under* O_{default}*, there exist properties* φ *and traces* $w \in \varphi$ *with* $\text{Good}(\varphi) \neq \emptyset$ *but no finite* n *such that* $V_\varphi(u_i^n) = T$*, where* u_i^n *is agent* i*'s observed prefix after* n *steps.*

Proof. Take $\varphi = \{w \in \Sigma^\omega : w \text{ contains } p\}$ (a cosafety property; $\text{Good}(\varphi)$ contains any prefix with a p). Let $w = p\sigma_1\sigma_2\ldots$ where p occurs only at step 0. If agent i's protocol never fires R_i at step 0 (for instance, R_i fires only at even steps ≥ 2), then u_i contains no p for any n; $V_\varphi(u_i^n) = U$ for all n even though $w \in \varphi$ and $\text{Good}(\varphi) \neq \emptyset$. □

Remark 10.2 (Extension pointer)**.** Closing this gap requires an *observation coverage* assumption — a fairness-like requirement that every event eventually enters some agent's observed prefix. Such an assumption belongs to an observation-kernel layer, not to the foundational core. The programme's implication/monitor-engineering line supplies one candidate kernel; adopting it as an extension is future work.

11 Related work

The paper stands on well-established shoulders. We summarise its positioning cluster by cluster.

Persistent and transient carriers. That a durable shared store supports coordination a bounded-lifetime message channel does not is standard, and we claim none of it. It is the premise of blackboard architectures [9], of stigmergic coordination through a modified environment, and of the persistent-log designs of distributed systems. Proposition 7.4 imports the fact and does one thing with it: it locates the asymmetry in the *reflexive* case, where Remark 7.1 does not reach, and derives it from the declared write footprints of Definition 4.2 together with Remark 6.3 rather than from a transport assumption. The novelty claimed is the placement, not the phenomenon.

Runtime verification and finite-prefix semantics. The prefix-verdict semantics of Section 5 is verbatim Bauer–Leucker–Schallhart RV-LTL [4], with the good-prefix / bad-prefix formulation traceable to Kupferman–Vardi [12]. The Alpern–Schneider dichotomy [1] classifies our target properties as safety, cosafety, and their complements; the four-class canon in RV form we use is a well-known runtime- verification convention rather than the six-class Manna–Pnueli hierarchy [13]. Theorems 5.1 and 9.1 are standard in this literature; we cite rather than derive.

Effect systems, non-interference, and dependency analysis. Theorem 6.2 is the classic "write-set disjoint from dependency set implies invariance" statement; footprint-based reasoning of this shape appears in abstract interpretation [8], in the interference-free rules of Owicki–Gries [16], and pervasively in effect systems, information-flow control, and separation logic. We adopt this discipline as machinery rather than claim it as contribution.

Labeled transition systems and process algebras. The many-sorted transition system of Section 3 is a minimal instance of the standard LTS pattern. Send / receive factorisation (Remarks 7.1 and 7.2) is standard in CCS [14] and CSP [11]; the statement here specialises it to a two-carrier communication model (M, P) typed by persistence.

Interpreted systems, epistemic and dynamic epistemic logic. Fagin, Halpern, Moses, and Vardi's interpreted systems [10] offer a richer Kripke-frame epistemic layer than the minimal (u_i, η_i) we adopt, but share the discipline that agent knowledge is a function of local view. Our verdict layer is a prefix classification, not a Kripke- frame knowledge modality; the distinction is deliberate and lets the framework compose with dynamic epistemic logic as an orthogonal extension. Remark 6.3 is the analogue of "no learning without perception" at the level of RV verdicts.

Agent architectures and action taxonomies. Classical sense-plan-act, sense-think-act, and OODA architectures distinguish perception from computation from action but rarely type communication as a separate primitive. The BDI architecture [17] adds mental attitudes; TAAS deliberately does not. Agent communication languages (FIPA, KQML) type speech-acts by illocutionary force rather than by state-component write set. Compared with these, TAAS is a smaller taxonomy but typed by footprint, and it splits transient (M) from persistent (P) communication at the primitive level — a distinction usually made only in shared-memory versus message-passing architectures, not at the action-primitive level.

Distributed / decentralised monitoring, diagnosability. The observation-coverage gap of Proposition 10.1 is well-studied in diagnosability theory [19] and in decentralised runtime verification [2]. The specific mathematical content of Proposition 10.1 is a diagnosability failure; the framework-level contribution is to name the gap as a first-class scope boundary and route extensions through it.

Separation logic and local reasoning. Separation logic [18, 15] formalises reasoning about disjoint mutable state and local effects; the frame rule expresses precisely the write-set disjointness we use in Theorem 6.2. *Owned:* the frame rule and separation-logic assertion discipline. *Overlap:* the frame-rule form of Theorem 6.2. *Difference:* the present framework works at a fixed partition of top-level state components at layer L_1 (not general heap addresses) and its target is prefix-verdict monotonicity, not memory safety. We claim no separation-logic novelty; Theorem 6.2 is a standard consequence of the frame rule specialised to this partition.

Region systems and storage lifetime. Region-based memory management [20] introduces lifetime discipline for storage: regions have declared scope and objects live within them. Our L_M/L_P carrier lifetime discipline in Section 7.1 is structurally comparable at the level of "storage has a lifetime." *Owned:* the region-lifetime formalism. *Difference:* we do not deploy region inference or region polymorphism; we declare two carrier classes with fixed lifetime semantics as part of the framework's typing choice and analyse a specific observable consequence (reflexive reentry).

Effect systems. Effect systems [21] type computations by the side-effects they may perform. Our footprint typing is structurally related: each action carries a declared $(\mathrm{Rd}, \mathrm{Wr})$ pair. *Owned:* the general effect-typing discipline. *Difference:* we do not provide an effect inference system or an effect polymorphism calculus; our footprint pairs are declared at the framework layer and analysed for their prefix-verdict consequences, not their programming-language typing.

STIT and formal agency. STIT (see-to-it-that) modal action logics [5] formalise agency in a branching-time modal setting: agents are responsible for state transitions. *Owned:* the modal formalisation of action responsibility. *Difference:* the present framework does not commit to a branching-time modal semantics or a responsibility calculus; it types actions by footprint and analyses prefix-verdict effects. STIT and this framework describe different aspects of agent action; no formal reduction or embedding is claimed.

Blackboard systems and persistent shared state. Blackboard architectures [9] realise persistent shared external state as a coordination substrate. Our persistent-carrier P echoes this structure at the framework layer. *Owned:* the blackboard architecture and its persistent-communication semantics. *Difference:* we do not propose a blackboard architecture; P is one of two typed carriers in a footprint-typing framework, and the FA-REENTRY result concerns the effect of persistence on reflexive-reentry verdicts, not blackboard coordination.

Comparative positioning. Each cluster contributes machinery that TAAS reuses. The specific combination *footprint-typed action classes over a partitioned state + RV-LTL prefix verdicts + explicit transient–persistent communication split + observation-coverage boundary as a scope statement* does not, to our knowledge, appear as a single package in the reviewed literature. This combination is a consolidation contribution, not a novelty claim on any individual ingredient.

12 The three-action quotient and its boundary

The five footprint action classes at layer L_1 (Definition 4.2) support a natural three-class quotient used by programme applications that do not inspect the externalisation carrier. This section defines that quotient, gives the conservativity scope, and shows that Proposition 7.4 is a boundary counterexample.

Definition 12.1 (Three-action quotient)**.** Let $\mathcal{A}_5 = \{\textsc{Read}, \textsc{Think}, \textsc{Speak}, \textsc{Write}, \textsc{Act}\}$ be the five footprint action classes at L_1. The *three-action quotient* $q_{5\to3} : \mathcal{A}_5 \to \mathcal{A}_3$ collapses the three externalisation classes into a single class Respond:

$$q_{5\to3}(\textsc{Read}) = \textsc{Read}, \quad q_{5\to3}(\textsc{Think}) = \textsc{Think},$$

$$q_{5\to3}(\textsc{Speak}) = q_{5\to3}(\textsc{Write}) = q_{5\to3}(\textsc{Act}) = \textsc{Respond},$$

where $\mathcal{A}_3 = \{\textsc{Read}, \textsc{Think}, \textsc{Respond}\}$.

Remark 12.2 (Not a new agent architecture)**.** The three-class Read/Think/Respond decomposition is structurally comparable to standard perceive-reason-act and sense-plan-act agent cycles used in the deliberative and reactive-agent literatures; the BDI architecture [17] organises the deliberation side, and analogous groupings appear in reactive and hybrid agent architectures. No novelty is claimed for the three-class grouping. The quotient $q_{5\to3}$ is a programme/interface construction that identifies the coarsest externalisation register at which a property family sees only the verdict transition and not its carrier.

Remark 12.3 (Conservativity scope)**.** Let Φ be a property family whose verdicts are invariant under substitutions among Speak, Write, and Act (equivalently, whose evaluation ignores the specific externalisation carrier). Then the quotient $q_{5\to3}$ is verdict-preserving on Φ: for every execution $w \in \mathcal{A}_5^*$ and every $\varphi \in \Phi$, $V_\varphi(w) = V_\varphi(q_{5\to3}(w))$. This follows immediately from the definition of Φ's carrier-insensitivity and the alphabet projection of $q_{5\to3}$; no separate proof obligation is created.

Remark 12.4 (FA-REENTRY as boundary counterexample)**.** The conservativity of Remark 12.3 is scope-relative: it fails for property families that inspect the externalisation carrier. Proposition 7.4 is the paradigmatic counterexample. In the reflexive-reentry setup with carrier lifetimes $L_M < \infty \leq L_P$ and delay $\Delta > L_M$, the composite $S_i(i); R_i$ (transient carrier, via SPEAK) and $W_i; R_i$ (persistent carrier, via WRITE) diverge at verdict layer even though $q_{5\to3}(S_i(i)) = q_{5\to3}(W_i) = \text{RESPOND}$. Coarse equality under $\mathcal{A}_3$ does not preserve the reentry property, and the exchange-of-lifetimes counterfactual (Remark 7.6) confirms the separation follows the carrier lifetime property, not the linguistic label SPEAK vs WRITE. This is the principal reason $\mathcal{A}_5$ is retained by this framework where the programme's carrier-insensitive applications use $\mathcal{A}_3$ safely.

Remark 12.5 (Programme application: LTL Triangles)**.** One programme application working within the carrier-insensitive scope of Remark 12.3 is the LTL Triangles formalisation of Berne transactional games and Karpman role projection, whose property family evaluates finite prefixes of role traces without inspecting the externalisation carrier of the intervention step. That work uses $\mathcal{A}_3$ as its operational-realisation vocabulary while citing Proposition 7.4 as the boundary discipline. The dependency direction is from LTL Triangles to Five Actions (LTL Triangles cites the quotient + boundary from this Technical Report); this Technical Report does not depend on LTL Triangles.

13 Temporal typing of capability and reentry properties

This section records how simple capability-availability and reentry state predicates lift to ω-properties under $\mathsf{S}/\mathsf{C}/\mathsf{L}/\mathsf{K}$ typing. It is a systematisation, not a new theorem; the machinery is imported verbatim from [7] and no new capability ontology is introduced.

Capability availability. For each L_1 footprint class $\text{Cls} \in \{\text{READ}, \text{THINK}, \text{SPEAK}, \text{WRITE}, \text{ACT}\}$ write $\text{Avail}_{\text{Cls}}(a, t)$ for the boolean holding when agent a is capable of a Cls-step at time t. Fix the base predicate $p = \text{Avail}_{\text{Cls}}(a, \cdot)$. Its canonical temporal lifts have distinct $\chi(\Phi) = (\mathbf{1}[\Phi \in \mathsf{S}], \mathbf{1}[\Phi \in \mathsf{C}], \mathbf{1}[\Phi \in \mathsf{L}], \mathbf{1}[\Phi \in \mathsf{K}])$ indicator vectors:

- $G\,p \in \mathsf{S} \cap \mathsf{K}$: capability is retained forever (finite bad prefix witnessed by any loss, no finite good prefix ever certifies it forever), minimum verdict rung $n = 3$;
- $F\,p \in \mathsf{C} \cap \mathsf{L}$: capability is eventually available (finite good prefix on first occurrence, no finite bad prefix), $n = 3$;
- $GF\,p \in \mathsf{L} \cap \mathsf{K}$ (recurrence, Π^0_2): capability is regained infinitely often, $n = 4$;
- $FG\,p \in \mathsf{L} \cap \mathsf{K}$ (persistence, Σ^0_2): capability stabilises after finitely many losses, $n = 4$.

The same base predicate populates three distinct cells under four different lifts. This is the standard four-family classification of [7], applied here to the footprint action classes of $\mathcal{A}_5$.

Recognition-response as the principal bridge property. For Rec a recognition event and Resp a response event realised through one or more of SPEAK, WRITE, ACT, the property

$$\Phi_{\text{RR}} \;=\; G\,(\text{Rec} \to F\,\text{Resp})$$

has $\chi(\Phi_{\text{RR}}) = (0, 0, 1, 1)$, cell $\mathsf{L} \cap \mathsf{K}$, minimum verdict rung $n = 4$. This is the generic recognition-to-response temporal type used by programme applications that identify SPEAK, WRITE, ACT through the quotient $q_{5\to3}$ (Remark 12.3). Because the recognition-response formula does not inspect the externalisation carrier, this family is q-invariant and passes through the three-action quotient unchanged.

FA-REENTRY: event formula vs architectural claim. Two distinct objects share the name reentry and are worth keeping formally separate. The *event formula*

$$\Phi_{\text{REENTRY},C} \;=\; F\big(\textsc{Speak}_C \wedge F\,\textsc{Read}_C\big)$$

per carrier $C \in \{M, P\}$ has $\chi = (0, 1, 1, 0)$, cell $\mathsf{C} \cap \mathsf{L}$, and minimum verdict rung $n = 3$, on the same footing as any other cosafety-monitorable formula. The *architectural reentry result* Proposition 7.4 additionally requires the carrier-lifetime clause $\Delta \leq L_C$ and the persistence relation $L_P \geq L_M$ from Definition 4.2; that clause is a stronger monitorability constraint that is not encoded in the LTL formula alone. Confusing the event with the architectural claim over-reads what the χ-signature commits to; the two objects sit at different layers and are only equivalent under implicit carrier assumptions.

$\mathcal{A}_5 \to \mathcal{A}_3$ preservation boundary. Temporal properties that do not mention the carrier are $q_{5\to3}$-invariant: their χ-signature is preserved under the quotient (recognition-response above is one such family, since Resp can be realised by any of Speak, Write, Act without carrier-specific commitment). Temporal properties whose LTL syntax mentions a specific carrier (such as $\Phi_{\text{REENTRY},C}$ above) are not $q_{5\to3}$-invariant and land at the boundary of Remark 12.3. This is a definitional consequence of $q_{5\to3}$ collapsing Speak, Write, Act into Respond; it is not proved as a theorem here.

Certified stopping and the temporal-class / refinement-confirmability orthogonality. An orthogonal certification layer for the persistence family $FG\,C$ is available when the internal Think-side computation is organised as a refinement system $\mathcal{R} = (D, A_0, U, E)$ in the sense of [6], with strict-decrease constant $c > 0$, bounded initial error $E(A_0) \leq E_{\max}$, and absorption of $C(A) \Leftrightarrow E(A) = 0$. Under those hypotheses Theorem 5.5 of [6] gives $k^* \leq \lceil E_{\max}/c \rceil$ with $E(A_{k^*}) = 0$, and Proposition 5.6(ii) upgrades to $\tau \models FG\,C$ under absorption; the agent may stop the refinement / experimentation loop relative to the declared target once $E(A_{k^*}) = 0$ has been finitely certified, relative to the admitted trace class $\mathcal{T}$. Refinement is an internal Think effect and not a sixth action class: no new footprint action is introduced. Two non-serial applications compose naturally with the framework of this paper. First, an externalisation bridge: once a finite certificate is obtained, Write on a persistent carrier P materialises the certificate, and Read later reacquires it; under $L_P \geq \Delta$ the certificate remains available across the delay (Proposition 7.4), so the RC-produced certificate composes with the FA-REENTRY architectural result without collapsing their theorem claims. Second, a system-level reading of the recognition- response family: the $\mathsf{L} \cap \mathsf{K}$-persistence view of the response mechanism itself, formalised as a refinement of the response-generation system with strict-decrease against "no pending unresponded Rec", is an RC target under the same hypotheses; the per-instance discharge of a single Rec obligation, by contrast, is ordinary bounded Cosafety on a finite prefix and does not require refinement confirmation.

Two boundaries deserve explicit statement. First, temporal-class membership and refinement-confirmability are orthogonal descriptors: recurrence $GF\,p$ and persistence $FG\,p$ share indicator vector $\chi = (0, 0, 1, 1)$ yet under [6] Proposition 5.6(iii) the recurrence form is *not* confirmed by finite descent without an explicit re-entry hypothesis, while persistence is confirmed under absorption. Second, a correct refinement argument relative to the wrong admitted trace class $\mathcal{T}$ does not certify the intended real-world property; the observation model and the admitted trace class are separate objects and identifying them would over-read the certificate. Generic empirical convergence of belief, statistical estimate, or learned parameter is not an instance of this scheme unless the exact strict-decrease structure with a fixed progress quantum $c > 0$ is exhibited.

14 Limitations

We list what the framework does not cover.

- The five action classes are the framework's declared L_1-footprint classes, not proposed as minimal or complete. Finer decompositions of READ by source and coarser groupings of communication by persistence exist.
- Only $O_{\text{default}} = (\text{perfect}, \text{non-invasive})$ is treated. Imperfect / invasive regimes require an observation kernel and are deferred.
- No causal / interventional effect sign C_φ is introduced. Continuation-set refinement, residual-property ordering, and controllability preorder each carry subtleties incompatible with a light foundational scope.
- No epistemic model K_i (beliefs, knowledge), no goals G_i, no policy family π_i beyond a type signature. These are extension points, not part of the core.
- No archetype convergence theorem $\alpha = \lim_{t\to\infty} P_t$ is proved. Convergence conditions are unspecified.
- No completeness or minimality result for the five-class taxonomy is claimed or proved.
- No general monitorability theorem: the framework relies on the standard RV monitorability theory rather than reformulating it.
- Certificate-acquisition guarantees (Proposition 10.1) require an observation-coverage assumption not stated here.

15 Conclusion

The framework asks for less than most agent frameworks and delivers correspondingly less: a typed integration interface between action classes and standard prefix-verdict monitor semantics. Its statement-level content is honest about lineage. Remark 6.3 is the analogue of "no learning without perception" at the RV verdict level, arising directly from the design choice that V_φ depends only on u_i. Remark 7.1 is standard send / receive factorisation, refined by the framework's transient-versus-persistent carrier split. Remark 8.1 enumerates the three reachable effect pairs and, in doing so, demotes a piece of notation. Proposition 10.1 names a diagnosability failure — known mathematical content — as a first-class scope boundary of the framework, so downstream extensions can name their observation- coverage assumption instead of leaving it implicit.

The framework has room to grow along four axes: the observation kernel for imperfect regimes; a causal sign C_φ once a clean formalisation is committed to; a strategic layer over certificate acquisition; and downstream applications — Karpman role dynamics, monitor engineering, mission-shaped multi-agent protocols — where the interface pays for itself. Each of these is a companion paper.

A Footprint table (full)

The following table restates Definition 4.2 explicitly at L_1 under O_{default}, and adds a persistence column useful for downstream extension.

Class	Rd_{L_1}	Wr_{L_1}	Direct verdict change?	Persistence	Direction
R_i	$\{E, M, P\}$	$\{X_i^{[u_i]}\}$ or $\{X_i^{[\eta_i]}\}$	yes (via u_i)	epistemic	self
T_i	$\{X_i\}$	$\{X_i^{[\eta_i]}\}$	no	epistemic	self
$S_i(j)$	$\{X_i\}$	$\{M\}$	no	transient	$i \to j$
W_i	$\{X_i\}$	$\{P\}$	no	persistent	self $\to$ public
A_i	$\{X_i, E\}$	$\{E\}$	no	permanent	$i \to$ environment

B Checker witnesses (MC1–MC15)

The checker script `checker/verify.py` verifies fifteen witnesses by exhaustive enumeration over a small finite alphabet. Every test is deterministic and terminates in constant time.

Test	Witness
MC1	Verdict absorption: $T \to T$, $F \to F$ on all extensions
MC2	Allowed 5-edge transition graph: $\{(U,U),(U,T),(U,F),(T,T),(F,F)\}$
MC3	Footprint non-interference: $\mathrm{Wr}(a) \cap \mathrm{Dep}(V_\varphi) = \emptyset \Rightarrow V_\varphi$ unchanged
MC4	READ direct verdict gate: only R can change $V_\varphi(u_i)$ in one step
MC5	$S(j); R_j$ transient communication composition witness
MC6	$W_i; R_k$ persistent communication composition witness
MC7	Three reachable (I_φ, R_φ) states
MC8	Liveness $U \to F$ impossibility (using Fp; Bad $= \emptyset$)
MC9	Coliveness $U \to T$ impossibility (using Gp; Good $= \emptyset$)
MC10	Certificate existence without acquisition (agent skipping R misses Good-prefix)
MC11	Reflexive reentry with $\Delta \leq L$: both carriers deliver the payload
MC12	Strictness for $\Delta > L$: transient path loses the signal, persistent path keeps it
MC13	Negative control: $L_M = \infty$ collapses the strictness
MC14	Negative control: $L_P < \Delta$ collapses the strictness
MC15	Reflexive read gate: $S_i(i)$, W_i, T_i leave $V_\varphi(u_i)$ fixed

All fifteen tests pass on the reference implementation.

References

[1] Bowen Alpern and Fred B. Schneider. Defining liveness. *Information Processing Letters*, 21(4):181–185, 1985.

[2] Andreas Bauer and Yliès Falcone. Decentralised LTL monitoring. *Formal Methods in System Design*, 48(1–2):46–93, 2016.

[3] Andreas Bauer, Martin Leucker, and Christian Schallhart. Comparing LTL semantics for runtime verification. *Journal of Logic and Computation*, 20(3):651–674, 2010.

[4] Andreas Bauer, Martin Leucker, and Christian Schallhart. Runtime verification for LTL and TLTL. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 20(4):14:1–14:64, 2011.

[5] Nuel Belnap. Backwards and forwards in the modal logic of agency. *Philosophy and Phenomenological Research*, 51(4):777–807, 1991. Cited as "Belnap 1993" after the follow-up STIT programme papers.

[6] Davide Bragetti. Refinement-based confirmation of liveness properties from monitor verdict traces: A Hensel–Newton approach. Zenodo, 2026. Concept DOI 10.5281/zenodo.22028018, v1 deposited 2026-08-20, CC BY 4.0.

[7] Davide Bragetti and Giuseppe Francesco Italiano. The intersection algebra of safety, cosafety, liveness, and coliveness over linear temporal logic. Zenodo, 2026. Version 2.

[8] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 238–252, 1977.

[9] Lee D. Erman, Frederick Hayes-Roth, Victor R. Lesser, and D. Raj Reddy. The Hearsay-II speech-understanding system: Integrating knowledge to resolve uncertainty. *ACM Computing Surveys*, 12(2):213–253, 1980.

[10] Ronald Fagin, Joseph Y. Halpern, Yoram Moses, and Moshe Y. Vardi. *Reasoning About Knowledge*. MIT Press, 1995.

[11] C. A. R. Hoare. *Communicating Sequential Processes*. Prentice-Hall, 1985.

[12] Orna Kupferman and Moshe Y. Vardi. Model checking of safety properties. *Formal Methods in System Design*, 19(3):291–314, 2001.

[13] Zohar Manna and Amir Pnueli. A hierarchy of temporal properties. In *Proceedings of the Ninth Annual ACM Symposium on Principles of Distributed Computing (PODC)*, pages 377–410, 1990.

[14] Robin Milner. *A Calculus of Communicating Systems*, volume 92 of *Lecture Notes in Computer Science*. Springer, 1980.

[15] Peter O'Hearn, John Reynolds, and Hongseok Yang. Local reasoning about programs that alter data structures. In *Computer Science Logic (CSL)*, pages 1–19, 2001.

[16] Susan Owicki and David Gries. An axiomatic proof technique for parallel programs I. *Acta Informatica*, 6(4):319–340, 1976.

[17] Anand S. Rao and Michael P. Georgeff. Modeling rational agents within a BDI-architecture. In *Proceedings of the 2nd International Conference on Principles of Knowledge Representation and Reasoning (KR)*, pages 473–484, 1991.

[18] John C. Reynolds. Separation logic: A logic for shared mutable data structures. In *Proceedings of the 17th Annual IEEE Symposium on Logic in Computer Science (LICS)*, pages 55–74, 2002.

[19] Meera Sampath, Raja Sengupta, Stéphane Lafortune, Kasim Sinnamohideen, and Demosthenis Teneketzis. Diagnosability of discrete-event systems. *IEEE Transactions on Automatic Control*, 40(9):1555–1575, 1995.

[20] Mads Tofte and Jean-Pierre Talpin. Region-based memory management. *Information and Computation*, 132(2):109–176, 1997.

[21] Philip Wadler. The marriage of effects and monads. In *Proceedings of the Third ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 63–74, 1998.